

Manual de Operación y Automatización del Observatorio

Basado en RTS2 (Remote Telescope System 2)

Universidad Politécnica de Madrid
Escuela Técnica Superior de Ingenieros Informáticos (ETSIINF)

18 de mayo de 2026

Índice

1. Introducción	2
2. Arquitectura del Sistema RTS2	3
2.1. Componentes y Demonios Principales	3
3. Configuración del Entorno (Linux)	4
3.1. Archivo <code>rts2.ini</code>	4
3.2. Configuración del Telescopio (Meade LX200)	4
3.3. Configuración de la Cúpula (Talon 6)	4
4. Arranque y Operación	5
4.1. Secuencia de Inicio	5
4.2. Interfaz de Usuario (<code>rts2-mon</code>)	5
5. Solución de Problemas Frecuentes (Troubleshooting)	6

1. Introducción

El presente documento constituye el Manual Oficial de Operación y Automatización del observatorio astronómico de la Escuela Técnica Superior de Ingenieros Informáticos (ETSIINF) de la Universidad Politécnica de Madrid (UPM).

Este proyecto nace de la iniciativa de un grupo de estudiantes con el objetivo de reparar, actualizar y automatizar las instalaciones del observatorio, el cual cuenta como instrumento principal con un telescopio **Meade LX200** y una cúpula motorizada controlada por el sistema **Talon 6**.

Para llevar a cabo esta automatización, se ha adoptado el software **RTS2** (Remote Telescope System 2), un entorno de código abierto ampliamente utilizado a nivel mundial para el control robótico de observatorios astronómicos. RTS2 permite no solo coordinar el movimiento del telescopio y la apertura de la cúpula, sino también programar secuencias de observación desatendidas, gestionar cámaras y registrar condiciones meteorológicas.

El objetivo principal de esta guía es proporcionar a los futuros alumnos e investigadores de la facultad un recurso claro, técnico y estructurado. Aquí se detalla desde la configuración inicial en sistemas Linux y la compilación del código fuente, hasta los comandos diarios de operación y la resolución de problemas (*troubleshooting*).

2. Arquitectura del Sistema RTS2

RTS2 está diseñado como un sistema **distribuido y modular** basado en *demonios* (procesos que se ejecutan en segundo plano en Linux). Esta arquitectura permite aislar los fallos: si la cámara falla, su demonio se reinicia sin afectar al seguimiento del telescopio ni a otros componentes.

Todos los demonios se comunican entre sí enviando mensajes a través del demonio central (**centrald**), que escucha habitualmente en el **puerto 8889** de la red local.

2.1. Arquitectura en Capas

El sistema RTS2 puede entenderse como una pila de cuatro capas funcionales:

Capa de Hardware

Los dispositivos físicos: telescopio, cúpula, cámara, focalizador y sensores. Cada uno se comunica con el PC mediante un protocolo propio (RS232, USB, red).

Capa de Demonios

Un proceso independiente por dispositivo traduce los comandos internos de RTS2 al protocolo del hardware. Si un demonio cae, el resto del sistema sigue funcionando.

Capa Central **centrald** actúa como enrutador de mensajes y árbitro del estado global. Es el único componente indispensable: sin él, ningún otro demonio puede comunicarse.

Capa de Automatización e Interfaces

Herramientas de alto nivel que orquestan la noche de observación (**execd**, **rts2-scheduler**) y permiten la interacción humana (**rts2-mon**, interfaz web HTTP).

2.2. Demonios de Dispositivo

Cada pieza de hardware tiene un demonio dedicado. Los disponibles en RTS2 son:

- **rts2-teld-lx200** — Demonio del telescopio para monturas con protocolo Meade LX200. Gestiona el apuntado (*slewing*), el seguimiento sidéreo y el modelo de alineación.
- **rts2-dome** — Demonio de la cúpula. Coordina la apertura de la rendija con la posición del telescopio para que siempre estén alineados. Se conecta al controlador **Talon 6** mediante `/dev/ttyS0`.
- **rts2-camd** — Demonio de la cámara CCD. Gestiona exposiciones, refrigeración y la escritura de las imágenes en formato FITS. (*Pendiente de instalación del hardware*).
- **rts2-focusd** — Demonio del focalizador motorizado. Permite enfocar automáticamente usando algoritmos de análisis de imagen. (*Pendiente*).

- **rts2-filterd** — Demonio de la rueda de filtros. Controla el cambio de filtro fotométrico entre exposiciones. (*Pendiente*).
- **rts2-sensord** — Demonio de sensores meteorológicos. Monitoriza nubes, lluvia, humedad y viento para cerrar el observatorio en caso de mal tiempo.

2.3. Demonios de Automatización y Control

- **centrald** — Enrutador central de mensajes (puerto 8889). Conoce el estado global del sistema y el modo de operación actual. Debe arrancarse siempre el primero.
- **execd** (Execution Daemon) — Lee los planes de observación y envía las órdenes en secuencia al resto del sistema: apuntar el telescopio, abrir la cúpula, capturar con la cámara, cambiar filtro.
- **rts2-scheduler** — Planificador automático de noches. Consulta la base de datos de objetivos y decide qué observar en cada momento según criterios de altitud, prioridad y tiempo disponible.
- **moodd** (Mode Daemon) — Gestiona el estado global del observatorio (ver Sección ??). Determina si el sistema está en modo diurno, crepuscular, nocturno o de emergencia.
- **rts2-logger** — Registra en la base de datos PostgreSQL todos los eventos del sistema: comandos, errores, resultados de observación y condiciones meteorológicas.
- **rts2-httpd** — Servidor HTTP que expone una API JSON/XML-RPC. Permite controlar el observatorio desde un navegador web o desde scripts Python remotos.

2.4. Base de Datos y Bibliotecas

PostgreSQL es el almacén central de RTS2. Guarda:

- El catálogo de objetivos astronómicos (*targets*) con sus coordenadas y restricciones de observación.
- El historial de observaciones realizadas (qué se observó, cuándo, con qué instrumento).
- Los metadatos de las imágenes FITS obtenidas.
- Los registros de eventos y alertas del sistema.

Internamente, RTS2 está construido sobre un conjunto de **bibliotecas C++11** que los demonios comparten:

Biblioteca	Función
<code>librts2</code>	Clases base: conexiones, variables compartidas, bucle de eventos.
<code>librts2db</code>	Acceso a PostgreSQL: targets, observaciones, logs.
<code>librts2tel</code>	Cálculos astrométricos: conversión de coordenadas, refracción.
<code>librts2script</code>	Motor de scripting de planes de observación.
<code>librts2fits</code>	Lectura y escritura de imágenes en formato FITS.
<code>librts2lx200</code>	Implementación del protocolo serie Meade LX200.

2.5. Modos de Operación del Sistema

`moodd` mantiene el sistema en uno de los siguientes estados. Cada modo habilita o bloquea acciones concretas (p.ej., en modo DAY no se puede abrir la cúpula):

Modo	Descripción
DAY	Operación diurna. La cúpula permanece cerrada. Se permiten calibraciones internas y mantenimiento.
DUSK	Crepúsculo vespertino. El sistema prepara los instrumentos: enfriamiento de la cámara, <i>flat fields</i> de cielo.
NIGHT	Noche astronómica. Modo de observación plena; el planificador (<code>rts2-scheduler</code>) toma el control.
DAWN	Amanecer astronómico. Fin de observaciones; se realizan <i>flat fields</i> de amanecer y se cierra la cúpula.
STANDBY	Emergencia meteorológica (lluvia, viento excesivo, nubes densas). El sistema cierra la cúpula de forma automática y espera.

Las transiciones entre modos las gestiona `moodd` de forma automática en función de la hora solar y de las lecturas de los sensores meteorológicos.

2.6. Flujo de una Observación Automática

Cuando el sistema opera en modo NIGHT con `execd` y `rts2-scheduler` activos, el ciclo de observación de un único objetivo transcurre así:

1. **Planificación:** `rts2-scheduler` consulta la base de datos y selecciona el siguiente objetivo según altitud, prioridad y tiempo restante de la noche.
2. **Asignación:** `rts2-scheduler` envía el objetivo a `execd`.
3. **Apuntado:** `execd` ordena a `centrald` que mueva el telescopio. `centrald` retransmite la orden a `rts2-teld-lx200`, que la ejecuta mediante el protocolo LX200 (`/dev/ttyUSB0`).
4. **Apertura de la cúpula:** `execd` ordena a `rts2-dome` que abra y alinee la rendija con la posición del telescopio. La cúpula sigue al telescopio durante toda la observación.

5. **Exposición:** Una vez el telescopio está en seguimiento, `execd` ordena a `rts2-camd` una exposición con el filtro y el tiempo indicados en el plan.
6. **Escritura:** `rts2-camd` guarda la imagen en formato FITS y `rts2-logger` registra los metadatos en PostgreSQL.
7. **Iteración:** `execd` señala a `rts2-scheduler` que el objetivo ha sido completado y el ciclo vuelve al paso 1.

3. Conexiones Físicas del Hardware

La siguiente tabla resume el estado actual de los dispositivos físicos instalados en el observatorio:

Componente	Modelo	Puerto (Win)	Puerto (Linux)	Estado
Telescopio	Meade LX200	COM3	/dev/ttyUSB0	Conectado
Cúpula	Talon 6	COM1	/dev/ttyS0	Conectado
Cámara CCD	Por determinar	—	—	Pendiente
Focalizador	Por determinar	—	—	Pendiente
Sensor meteo	Por determinar	—	—	Pendiente

Las conexiones físicas actuales son:

- **Telescopio Meade LX200:** el PC se conecta mediante un **adaptador USB–RS232**. En Linux, el adaptador aparece como /dev/ttyUSB0 (velocidad 9600 bps, sin paridad, 8 bits de datos).
- **Cúpula Talon 6:** conectada al **puerto serie nativo** del PC (/dev/ttyS0). No requiere adaptador.
- **Futuros instrumentos** (cámara, focalizador): se conectarán probablemente mediante USB o FireWire, y se integrarán a través del demonio `rts2-camd` correspondiente al modelo elegido.

4. Configuración del Entorno (Linux)

Los archivos de configuración principales de RTS2 se alojan en el directorio `/etc/rts2/`.

4.1. Archivo `rts2.ini`

(En esta sección se documentará el contenido del archivo principal extraído del ordenador del laboratorio).

4.2. Configuración del Telescopio (Meade LX200)

El Meade LX200 se comunica con el ordenador mediante un adaptador USB–RS232. En entornos Windows se correspondía históricamente con el puerto `COM3`, lo cual en Linux se traduce habitualmente en el dispositivo `/dev/ttyUSB0` (o `ttyUSB1`).

(Aquí se añadirán las líneas de configuración exactas una vez se tengan los archivos).

4.3. Configuración de la Cúpula (Talon 6)

El sistema de control de la cúpula Talon 6 se encuentra conectado por un puerto serie nativo. En Windows se mapea como `COM1`, lo que en sistemas Linux equivale directamente al dispositivo `/dev/ttyS0`.

(Aquí se añadirán las líneas de configuración exactas una vez se configure el driver).

5. Arranque y Operación

5.1. Secuencia de Inicio

El orden de arranque de los demonios es **estricto**: `centrald` debe estar activo antes de que cualquier otro demonio intente conectarse. El orden completo recomendado es:

1. **Base de datos** (prerequisito de todo el sistema):

```
$ sudo systemctl start postgresql
```

2. **Enrutador central** (siempre el primero):

```
$ rts2-centrald
```

3. **Demonio del telescopio**:

```
$ rts2-telld-lx200 -d /dev/ttyUSB0
```

4. **Demonio de la cúpula**:

```
$ rts2-dome -d /dev/ttyS0
```

5. **Demonio de ejecución** (opcional, para observación automática):

```
$ rts2-executor
```

6. **Interfaz de usuario**:

```
$ rts2-mon
```

Nota

En un entorno de producción final, estos servicios se configurarán para iniciar automáticamente en el arranque del sistema mediante **systemd**. El repositorio incluye plantillas de unit files en `conf/`.

5.2. Interfaz de Usuario (`rts2-mon`)

Para interactuar con el observatorio, ver el estado de los componentes y enviar comandos de forma manual, RTS2 incluye una consola interactiva llamada **rts2-mon**.

Al ejecutar **rts2-mon** en la terminal, se despliega una interfaz basada en texto (ncurses). **Importante**: el ratón no funciona en esta interfaz. Se deben usar las flechas del teclado para moverse entre las pestañas de cada dispositivo (Telescopio, Cúpula, Cámara, etc.) y la tecla **Enter** para interactuar.

Desde **rts2-mon** se puede, por ejemplo, introducir las coordenadas de Ascensión Recta (RA) y Declinación (Dec) y ordenar al Meade LX200 que comience a moverse hacia el objetivo (acción conocida como *slewing*).

5.3. Interfaz Web (rts2-httpd)

`rts2-httpd` expone una API HTTP/JSON en el puerto 8889 (por defecto). Permite:

- Consultar el estado de todos los demonios desde un navegador web.
- Enviar comandos de forma remota mediante peticiones HTTP.
- Integrar el observatorio con scripts Python usando el módulo `python3/rts2.py` incluido en el repositorio.

6. Solución de Problemas Frecuentes (*Troubleshooting*)

Dado que RTS2 se ejecuta en un entorno Linux, es habitual enfrentarse a ciertos errores de configuración. Esta sección sirve como guía de primeros auxilios.

■ El telescopio no responde a los comandos de movimiento:

1. ¿El PC reconoce el cable? Ejecutar en la terminal:

```
$ dmesg | grep tty
```

dmesg muestra los mensajes internos de hardware de Linux y confirma si el adaptador USB-RS232 se ha detectado como */dev/ttyUSB0* o similar.

2. ¿Hay permisos? En Linux, los programas no pueden leer puertos hardware sin permiso explícito:

```
$ sudo chmod a+rw /dev/ttyUSB0
```

(Sustituir *ttyUSB0* por el puerto detectado en el paso anterior).

3. ¿El demonio está vivo?

```
$ ps aux | grep rts2-teld
```

Lista los procesos en ejecución y confirma si el demonio del telescopio sigue activo.

4. Consultar los logs:

```
$ tail -f /var/log/syslog
```

RTS2 guarda un diario de todos los eventos en */var/log/rts2/*.

■ Error de conexión al iniciar los demonios (*no encuentran centrald*):

Todos los módulos asumen que *centrald* ya está funcionando. Si *centrald* se cierra de golpe, el resto de demonios quedarán bloqueados intentando conectarse. La solución es parar todos los servicios y volver a levantarlos, arrancando *centrald* siempre en primer lugar.

■ Fallo general de RTS2 o errores al arrancar:

RTS2 necesita conectarse a PostgreSQL obligatoriamente. Comprobar el estado de la base de datos:

```
$ systemctl status postgresql
```

Si el estado indica *inactive* o *failed*, intentar reiniciarla:

```
$ sudo systemctl restart postgresql
```

■ La cúpula no sigue al telescopio:

Comprobar que el demonio *rts2-dome* está registrado en *centrald* (visible en *rts2-mon*). Verificar también que */dev/ttyS0* tiene permisos de lectura/escritura:

```
$ ls -l /dev/ttyS0  
$ sudo chmod a+rw /dev/ttyS0
```

- **La cúpula cierra automáticamente en modo standby:**

Este comportamiento es correcto: `moodd` ha detectado una alarma meteorológica (lluvia, viento, humedad). Revisar las lecturas de `rts2-sensord` en `rts2-mon` para identificar el sensor que ha disparado la alerta.